AICW AI & ML COMPANION

Advanced Machine Learning & Deep Learning

From a single neuron to a full image classifier, explained in plain English

Chapter 4 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. This reader walks beside your lecture slides. It never just names an idea — it explains it in plain words first. Five colours guide you through each topic: a **blue** box gives the core idea in plain English; an **amber** box gives a real-life comparison; a **green** box solves a real example, number by number; a **red** box flags a common mistake; and a **purple** box gives the one line to remember. Read a section start to finish, in order — each one builds on the last.

1 The Neuron: The Atom of Deep Learning

How does a machine "decide" anything? Every huge model — the one that tags a photo, writes a sentence, or drives a car — is built from one tiny, repeated part: the *neuron*. If you understand this one part, you understand the brick that every deep learning wall is built from.

☞ The intuition

A neuron takes in some numbers, mixes them together using its own personal weighting, adds one fixed extra number, and then squashes the result through a bend. That squashed number is its output. Nothing more mysterious happens inside a neural network — it is this same small step, repeated millions of times.

★ Everyday picture

Think of a neuron as a strict but fair judge scoring a talent show. Each act (input) gets a score, but the judge cares about some acts more than others (weight). The judge also walks in with a personal mood that day (bias) — maybe grumpy, maybe generous — which shifts every score up or down a little before the final verdict (activation) is announced.

Building the formula, one piece at a time

Say the neuron receives n input numbers $x_1, x_2, \ldots, x_n$. Each input has its own **weight** $w_1, w_2, \ldots, w_n$ — a number that says how much that input matters. First, the neuron multiplies each input by its weight and adds them all up:

$$\text{weighted sum} \;=\; w_1x_1 + w_2x_2 + \cdots + w_nx_n.$$

Then it adds one more number called the **bias**, written b. The bias is not tied to any input — it is a fixed number the neuron always adds, shifting the result up or down:

$$z \;=\; w_1x_1 + w_2x_2 + \cdots + w_nx_n + b.$$

Finally, z passes through an **activation function**, written f, which bends the number into its final shape. The neuron's output is

$$y \;=\; f(z).$$

That is the whole neuron. We look closely at f in Section 2.

✎ Worked example: one neuron, three inputs

A neuron gets three inputs $x = [0.5,\ 0.6,\ 0.1]$, with weights $w = [0.2,\ 0.3,\ 0.9]$ and bias $b = 0.5$. It uses the sigmoid activation (defined fully in Section 2). Find its output y.

Step 1. Multiply each input by its weight.

$$0.2 \times 0.5 = 0.10, \qquad 0.3 \times 0.6 = 0.18, \qquad 0.9 \times 0.1 = 0.09.$$

Step 2. Add the three products together.

$$0.10 + 0.18 + 0.09 = 0.37.$$

Step 3. Add the bias.

$$z = 0.37 + 0.5 = 0.87.$$

Step 4. Apply the activation function. The sigmoid formula is $f(z) = 1/(1 + e^{-z})$. Plugging in $z = 0.87$:

$$y = \frac{1}{1 + e^{-0.87}} = \frac{1}{1 + 0.419} = \mathbf{0.7047}.$$

Step 5. Sense-check. The output sits between 0 and 1, as sigmoid always gives. A value of 0.70 reads as "fairly confident yes."

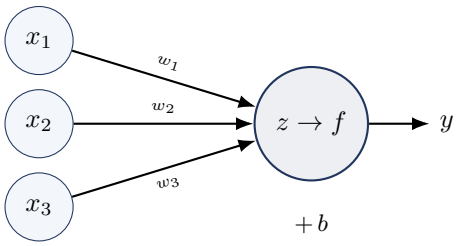


Figure 1. A single neuron: each input is scaled by its own weight, the bias is added once, and the sum is squashed by an activation function into the output y.

✗ Watch out

It is easy to forget the bias and think a neuron is just a weighted average. It is not — the bias lets the neuron shift its decision boundary even when every input is zero. It is also easy to assume weights must be positive; a negative weight simply means "this input pushes the answer the other way."

Where this shows up in ML. This exact recipe — multiply, add, add bias, squash — is the `Dense` layer in Keras/TensorFlow and the `Linear` layer in PyTorch. Every "fully connected" layer you will ever build is a row of these neurons running side by side.

✔ **Key takeaway**

A neuron = a weighted sum of its inputs, plus one bias, pushed through an activation function. Stack many of these in layers and you get a neural network.

2 Activation Functions: Adding the Bend

If a neuron only ever added and multiplied, then stacking a hundred of them would still just be one big multiplication — a straight line in disguise. Straight lines cannot draw a curve, and almost nothing in the real world is a straight line.

☞ The intuition

An activation function is a fixed rule that bends a neuron's number. Without it, a "deep" network could never learn anything a single straight line couldn't already do. With it, stacking layers lets the network bend and combine curves to fit almost any shape.

★ Everyday picture

A light with an old-fashioned on/off switch can only be fully off or fully on. A dimmer switch lets you set it anywhere in between, smoothly. The **sigmoid** activation acts like a dimmer switch: it turns any number into a smooth value between 0 and 1. The **ReLU** activation acts like a one-way valve in a pipe: water (a positive number) flows through unchanged, but any negative pressure is blocked and comes out as zero.

Two activation functions, built step by step

The **sigmoid** function takes any real number z and squashes it into the range $(0, 1)$:

$$\sigma(z) \;=\; \frac{1}{1+e^{-z}}.$$

Large positive z gives an output near 1; large negative z gives an output near 0; $z = 0$ gives exactly 0.5.

The **ReLU** function (Rectified Linear Unit) is much simpler. It keeps positive numbers and turns negative numbers into zero:

$$\mathrm{ReLU}(z) \;=\; \max(0, z).$$

✎ Worked example: comparing sigmoid and ReLU on the same numbers

Using $z = 0.87$ from Section 1's neuron, and two new test values, compare both activations.

Step 1. **Sigmoid at $z = 0.87$ (already found):** $\sigma(0.87) = 0.7047$.

Step 2. **ReLU at $z = 0.87$:** the number is already positive, so $\mathrm{ReLU}(0.87) = 0.87$ — unchanged.

Step 3. **ReLU at $z = -2$:** the number is negative, so $\mathrm{ReLU}(-2) = 0$.

Step 4. **Sigmoid saturation check at $z = 5$:** $\sigma(5) = 1/(1+e^{-5}) = 1/(1.0067) = 0.9933$. The slope there, $\sigma(z)(1-\sigma(z)) = 0.9933 \times 0.0067 = 0.0067$, is almost flat.

Step 5. **So what?** At large z, sigmoid's slope nearly vanishes — a problem we call *vanishing gradients* (Section 5). ReLU's slope stays a full 1 for any positive z, which is one reason it trains faster in deep nets.

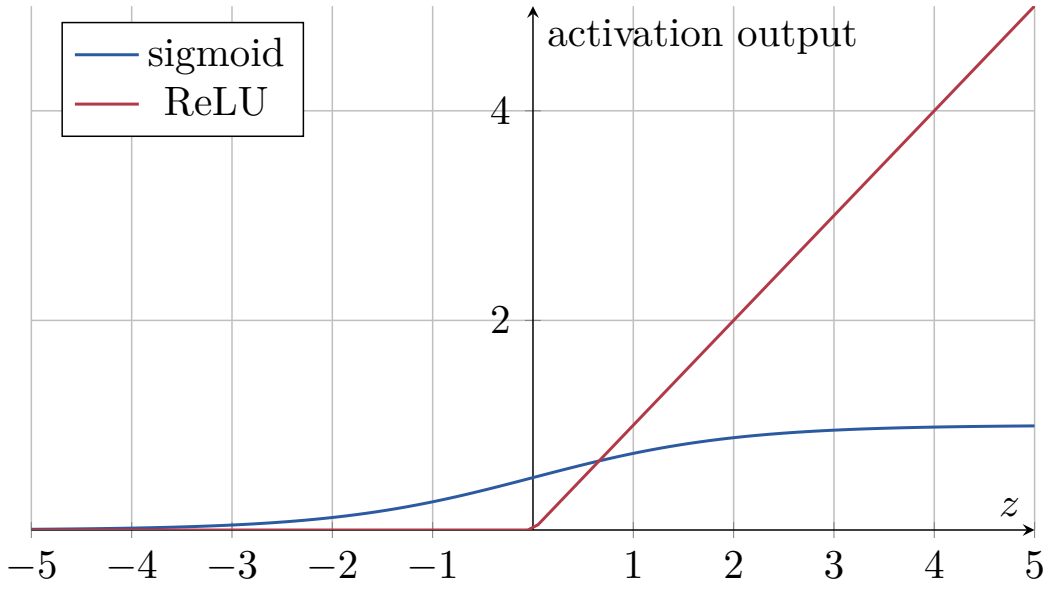


Figure 2. Sigmoid smoothly saturates near 0 and 1 — flat at the ends, where learning slows. ReLU stays a straight 45° line for $z > 0$ and is flat zero for $z < 0$.

✗ Watch out

Sigmoid outputs are always positive and centred around 0.5, not 0 — this can slow learning in deep hidden layers, which is why ReLU (or its cousins) is the default choice for hidden layers today. Sigmoid is still the right choice for a final layer producing a single probability.

Where this shows up in ML. The lecture slides name ReLU directly as the activation used inside convolution layers (Section 8). Sigmoid is used at the output of a binary classifier, and its close cousin, softmax, is used for multi-class outputs (Section 10).

✔ Key takeaway

Activation functions are what let a network bend, not just add. ReLU is the default for hidden layers; sigmoid (and softmax) are for turning final scores into probabilities.

3 Shallow vs. Deep Networks

☞ The intuition

A **shallow** network has only one or two hidden layers standing between its input and its output. A **deep** network stacks many hidden layers, each one building on the pattern the layer before it found. "Deep learning" is named after exactly this stacking.

★ Everyday picture

Picture a factory assembly line. A shallow line has one workstation that must do almost everything at once — cut, shape, and paint the part in one go. A deep line has many small, specialised stations: one only cuts, the next only shapes, the next only paints. Each station's job is simple, but together they build something complex. Deep networks work the same way — early layers might notice simple edges, later layers combine those edges into shapes, and even later layers combine shapes into whole objects.

✎ Worked example: counting the "knobs" in a shallow vs. a deep network

Compare a shallow network (input 3, one hidden layer of 4, output 2) to a deep network of the same width (input 3, three hidden layers of 4, output 2). A layer's own parameter count is (inputs × neurons) + one bias per neuron.

Step 1. **Shallow — hidden layer:** $3 \times 4 + 4 = 16$ parameters.

Step 2. **Shallow — output layer:** $4 \times 2 + 2 = 10$ parameters.

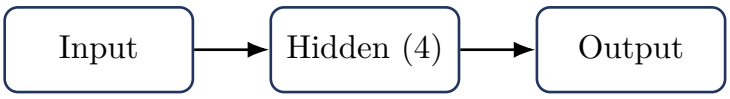
Step 3. **Shallow total:** $16 + 10 = \mathbf{26}$ parameters.

Step 4. **Deep — three hidden layers:** layer 1 is $3 \times 4 + 4 = 16$; layer 2 is $4 \times 4 + 4 = 20$; layer 3 is $4 \times 4 + 4 = 20$.

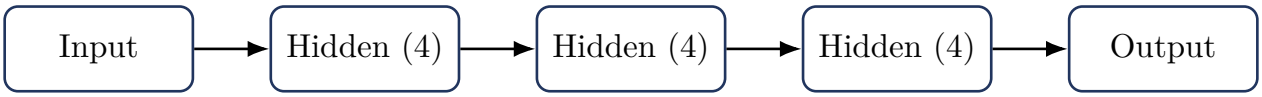
Step 5. **Deep — output layer:** $4 \times 2 + 2 = 10$ parameters.

Step 6. **Deep total:** $16 + 20 + 20 + 10 = \mathbf{66}$ parameters.

Step 7. **So what?** The deep network has more than twice the "knobs" to tune, which lets it represent far more complex functions — but it also needs more data to learn those knobs well, or it starts memorising instead of learning (Section 6).



Shallow network



Deep network

Figure 3. A shallow network has one hidden layer standing between input and output. A deep network stacks

several — each building on the last.

✗ Watch out

"Deeper is always better" is a myth. Extra depth without enough data or the right tricks (Sections 5, 7) can make training harder, not easier — this is the vanishing-gradient problem hinted at in Section 2.

Where this shows up in ML. Modern breakthroughs — image classifiers, translation systems, and large language models — are all deep by this exact definition, often with dozens or hundreds of stacked layers.

✔ Key takeaway

Depth is stacking hidden layers so each one refines the last layer's pattern. More depth means more representation power, but also more to train.

4 Forward Propagation: How a Network Makes a Guess

☞ The intuition

Forward propagation is simply running the neuron recipe from Section 1, layer after layer, left to right: inputs go in, each layer computes its outputs, and those outputs become the next layer's inputs — until a final answer comes out.

★ Everyday picture

It is exactly the factory line from Section 3: raw material (the input numbers) enters at one end, each station transforms it a little, and a finished product (the prediction) rolls off the other end. Nothing runs backward yet — that is Section 5's job.

Naming the pieces

For a network with one hidden layer, call the input vector x, the hidden layer's weights $W^{[1]}$ and bias $b^{[1]}$, and the output layer's weights $W^{[2]}$ and bias $b^{[2]}$. Forward propagation is two applications of the neuron recipe:

$$z^{[1]} = W^{[1]}x + b^{[1]}, \qquad a^{[1]} = \mathrm{ReLU}(z^{[1]}),$$
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}, \qquad y = \sigma(z^{[2]}).$$

Here $a^{[1]}$ (the hidden layer's activated output) becomes the input to the next layer — that is the whole "propagation forward" idea in two lines.

✎ Worked example: a full forward pass through a tiny network

A network has two inputs, two hidden neurons (ReLU), and one output neuron (sigmoid). Weights: h_1 uses $(0.6, -0.4)$ with bias 0.1; h_2 uses $(0.3, 0.8)$ with bias -0.2; the output neuron uses weights $(0.5, -0.9)$ on (h_1, h_2) with bias 0.05. Inputs are $x_1 = 1,\ x_2 = 0$. Find the output y.

Step 1. Hidden neuron 1. $z_{h_1} = 0.6(1) + (-0.4)(0) + 0.1 = 0.7$, so $h_1 = \mathrm{ReLU}(0.7) = 0.7$.

Step 2. Hidden neuron 2. $z_{h_2} = 0.3(1) + 0.8(0) - 0.2 = 0.1$, so $h_2 = \mathrm{ReLU}(0.1) = 0.1$.

Step 3. Output neuron's weighted sum. $z_{out} = 0.5(0.7) + (-0.9)(0.1) + 0.05 = 0.35 - 0.09 + 0.05 = 0.31$.

Step 4. Apply sigmoid. $y = \sigma(0.31) = 1/(1 + e^{-0.31}) = 1/1.7334 = \mathbf{0.5769}$.

Step 5. Sense-check. 0.577 sits just above 0.5 — a weak "yes." We reuse this exact pass in Section 5 to see how the network would correct itself.

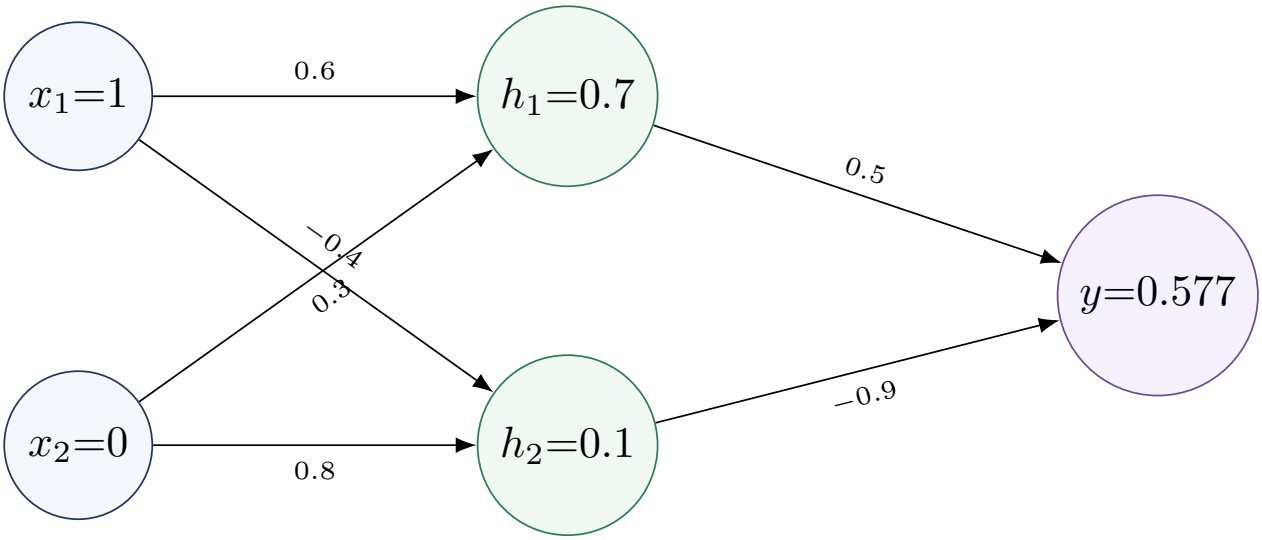


Figure 4. The forward pass: numbers flow left to right through the weighted edges, each node applying its activation, until a single output emerges.

✗ Watch out

Always compute z (the weighted sum) fully before applying the activation — a common slip is applying the activation to each term separately instead of to their sum.

Where this shows up in ML. This is exactly what `model.predict()` in Keras or `model.forward()` in PyTorch executes — layer after layer, using the exact weights the model has learned.

✔ Key takeaway

Forward propagation is the neuron recipe applied layer by layer, left to right, turning raw inputs into a prediction.

5 Backward Propagation: How a Network Learns from Its Mistakes

☞ The intuition

Forward propagation makes a guess. **Backward propagation** (backprop) is how the network learns from being wrong: it measures the error, then works backward through the layers assigning "blame" to each weight, telling each one exactly how much and which way to change.

★ Everyday picture

Picture a sports coach reviewing game tape after a loss. The coach does not just say "we lost" — they trace the loss back: this missed pass led to that turnover, which led to that goal. Each player gets specific, proportional feedback on what to change next time. Backprop does the same thing to every weight in the network.

Measuring error and stepping downhill

First we need a **loss function**, a number that says how wrong a prediction was. A common one is squared error:

$$L = \tfrac{1}{2}(y_{\text{true}} - y)^2.$$

Backprop uses the **chain rule** to find how much a tiny change in each weight would change L — this is called the weight's **gradient**, written $\partial L/\partial w$. We then nudge every weight a small step in the direction that reduces L:

$$w \;\leftarrow\; w - \eta\,\frac{\partial L}{\partial w},$$

where η (the Greek letter eta) is the **learning rate** — how big a step we dare to take.

✎ Worked example: one weight update, by hand

Reuse Section 4's forward pass ($h_1 = 0.7,\ h_2 = 0.1,\ y = 0.5769$). Suppose the true label was $y_{\text{true}} = 1$. Update the output weight $v_1 = 0.5$ (on h_1) with learning rate $\eta = 0.1$.

Step 1. Compute the loss. $L = \tfrac{1}{2}(1 - 0.5769)^2 = \tfrac{1}{2}(0.4231)^2 = \mathbf{0.0895}$.

Step 2. How loss changes with y. $\partial L/\partial y = -(y_{\text{true}} - y) = -(0.4231) = -0.4231$.

Step 3. How y changes with z_{out}. For sigmoid, $\partial y/\partial z_{out} = y(1-y) = 0.5769 \times 0.4231 = 0.2441$.

Step 4. Chain them together. $\partial L/\partial z_{out} = (-0.4231)(0.2441) = -0.1033$.

Step 5. How z_{out} changes with v_1. Since $z_{out} = v_1 h_1 + v_2 h_2 + c$, we get $\partial z_{out}/\partial v_1 = h_1 = 0.7$. So $\partial L/\partial v_1 = (-0.1033)(0.7) = \mathbf{-0.0723}$.

Step 6. Update the weight. $v_1^{\text{new}} = 0.5 - 0.1 \times (-0.0723) = 0.5 + 0.0072 = \mathbf{0.5072}$.

Step 7. Sense-check. The weight moved *up* slightly, which will push next time's output a little closer to the target of 1 — exactly the correction we wanted.

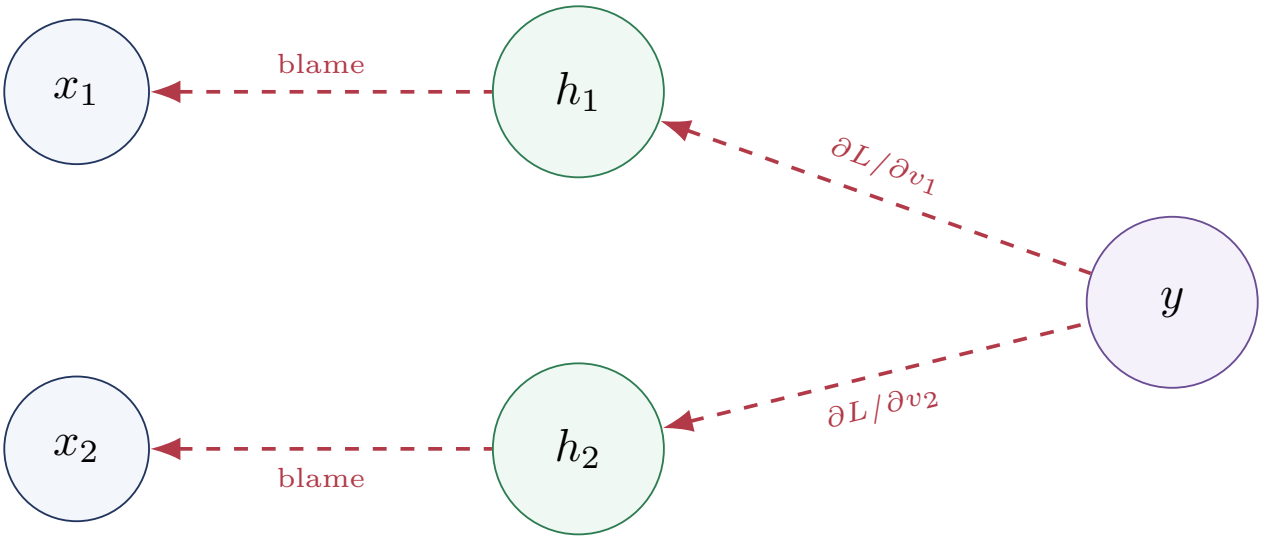


Figure 5. Backward propagation walks the same network in reverse, handing each weight a share of the blame for the error.

✗ Watch out

A learning rate that is too large overshoots the correction and can make the loss bounce around or even grow; too small, and training crawls. In very deep networks, gradients can also shrink to almost nothing as they travel backward — the same saturation problem flagged for sigmoid in Section 2.

Where this shows up in ML. This entire worked example is what `loss.backward()` followed by `optimizer.step()` does in PyTorch, or what TensorFlow's `GradientTape` computes automatically — you never do this arithmetic by hand in practice, but now you know exactly what is happening underneath.

✔ Key takeaway

Backpropagation applies the chain rule layer by layer to find each weight's share of the blame for the error, then gradient descent nudges every weight a small step to reduce that error.

6 Overfitting and Underfitting: When Learning Goes Wrong

☞ The intuition

A well-trained model should do well not just on the data it saw, but on new data it has never seen. **Overfitting** means the model memorised the training data — including its noise and quirks — so it fails on new data. **Underfitting** means the model never learned the pattern well enough, so it does poorly everywhere, even on the training data.

★ Everyday picture

Picture three students before a maths test. Student A barely pays attention in class and does poorly on both the daily quiz and the real exam — that is **underfitting**: they never learned the pattern at all. Student B memorises every practice question word for word, scoring almost perfectly on the daily quiz, but the real exam asks new questions in a new order — and Student B falls apart. That is **overfitting**: great on seen material, poor on new material. Student C studies the underlying method, not just the answers, and does reasonably well on both the quiz and the exam. Student C is what we want a model to be.

✎ Worked example: spotting overfitting from a training/test error table

Three trained models are tested on their training set ("quiz") and a held-out test set ("exam"). The **generalisation gap** is gap = test error − train error: a small gap means the model behaves the same on new data as on old data.

Step 1. Student A (underfit model): train error 58%, test error 60%. Gap $= 60 - 58 = 2$ — a small gap, but *both* numbers are high, so the model never learned the pattern.

Step 2. Student B (overfit model): train error 2%, test error 48%. Gap $= 48 - 2 = \mathbf{46}$ — a huge gap. It nearly aced the training data but fails badly on new data.

Step 3. Student C (good fit): train error 15%, test error 18%. Gap $= 18 - 15 = 3$ — small, and both numbers are reasonably low.

Model	Train error	Test error	Gap
Student A (underfit)	58%	60%	2
Student B (overfit)	2%	48%	46
Student C (good fit)	15%	18%	3

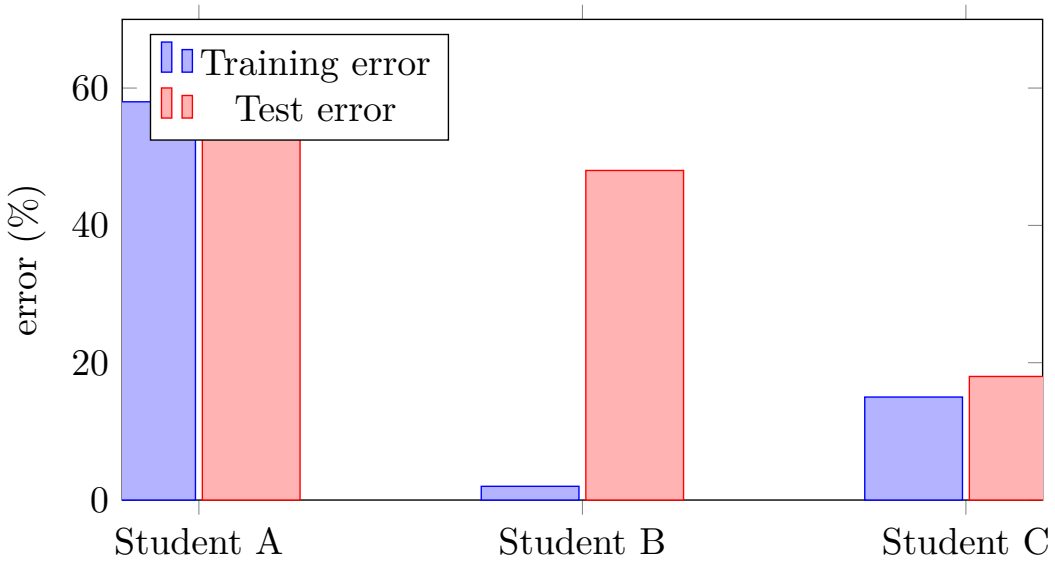


Figure 6. Student B's tiny training error paired with a huge test error is the signature shape of overfitting. Student C's two close, low bars are the goal.

✗ Watch out

Never judge a model by training accuracy alone — a model can look perfect on paper and still be useless in the real world. Always check performance on data the model has never seen.

Where this shows up in ML. This is exactly why every serious ML project holds out a validation or test set, and why cross-validation exists — to catch overfitting before it ships.

✔ Key takeaway

A good model has low error on both training and test data, with a small gap between the two. A big gap means overfitting; both numbers being high means underfitting.

7 Fighting Overfitting: Dropout and Early Stopping

☞ The intuition

Once we can detect overfitting (Section 6), we need tools to stop it. **Dropout** randomly switches off some neurons during training so the network cannot lean too heavily on any one of them. **Early stopping** watches the validation error during training and stops the moment it starts getting worse, even if the training error keeps improving.

★ Everyday picture

Dropout is like a football coach who, during practice only, randomly benches a few players each drill. No single player — not even the star — can become the team's only plan; everyone has to learn to contribute. **Early stopping** is like a student who checks their practice-exam score after every study session and stops studying a topic the moment their score plateaus or starts to dip, instead of endlessly re-cramming the same material.

Dropout, with real numbers

During training, dropout picks a **keep probability** p (say $p = 0.5$), randomly zeroes out some neurons' outputs, and — to keep the overall signal strength the same — scales up the surviving ones by $1/p$.

✎ Worked example: dropout on a layer of four neurons

A hidden layer outputs $[0.8,\ 0.3,\ 0.6,\ 0.9]$ before dropout. Using keep probability $p = 0.5$, this training pass happens to drop neuron 2.

Step 1. Apply the mask. Zero out the dropped neuron: $[0.8,\ 0,\ 0.6,\ 0.9]$.

Step 2. Scale the survivors by $1/p = 1/0.5 = 2$. $[0.8{\times}2,\ 0,\ 0.6{\times}2,\ 0.9{\times}2] = [\mathbf{1.6},\ 0,\ \mathbf{1.2},\ \mathbf{1.8}]$.

Step 3. So what? This scaled, partly-zeroed vector — not the original — is what the next layer sees during this training step.

Step 4. At test time, dropout is turned off entirely: the network uses the original $[0.8,\ 0.3,\ 0.6,\ 0.9]$ with no masking and no scaling, because the scaling during training already accounted for it.

Early stopping, with real numbers

✎ Worked example: finding the stopping point from a validation-loss table

A model is monitored with a patience of 2 epochs (stop once 2 epochs in a row fail to beat the best score so far).

Step 1. Record the validation loss each epoch: epoch 1: 0.62, epoch 2: 0.51, epoch 3: 0.47, epoch 4: 0.49, epoch 5: 0.52, epoch 6: 0.50.

Step 2. Track the best-so-far. The best loss is 0.47, reached at epoch 3.

Step 3. Count epochs without improvement. Epoch 4 (0.49) does not beat 0.47 — that is strike 1. Epoch 5 (0.52) also does not beat it — strike 2.

Step 4. Patience is used up. Training stops after epoch 5. The model keeps epoch 3's weights, since that was the best-ever validation score.

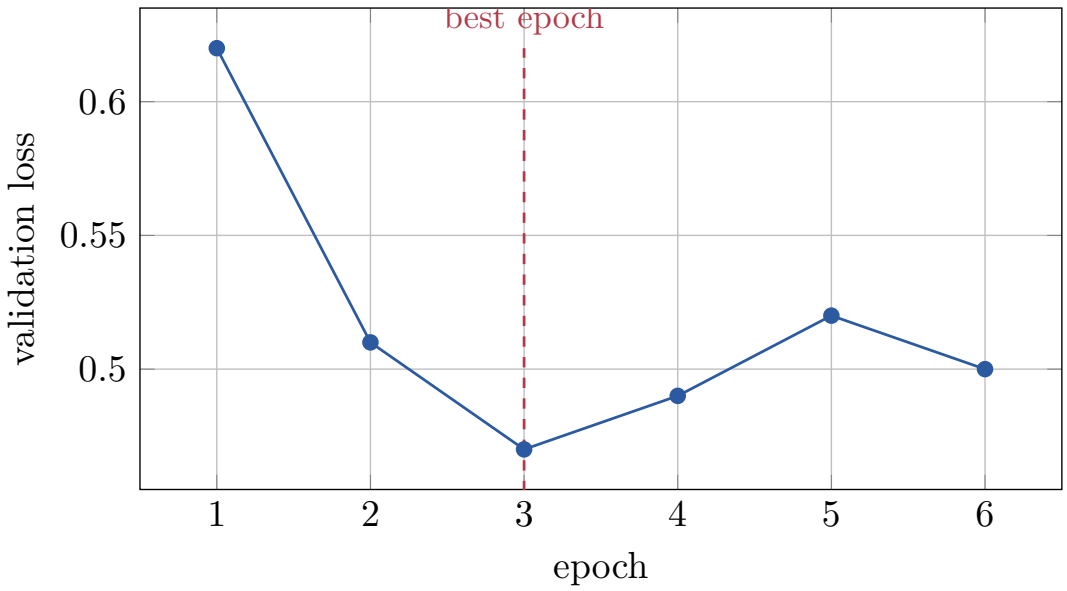


Figure 7. Validation loss bottoms out at epoch 3, then rises for two epochs straight — that is where early stopping steps in and keeps epoch 3's weights.

✗ Watch out

Dropout must be switched off at test time — leaving it on at inference makes predictions noisy and inconsistent. Setting the early-stopping patience too low can stop training before the model has really converged.

Where this shows up in ML. These are one-line tools in Keras: `tf.keras.layers.Dropout(0.5)` and `tf.keras.callbacks.EarlyStopping(patience=2)`.

✔ Key takeaway

Dropout forces a network to not rely on any single neuron; early stopping halts training at the moment new data stops improving, protecting against overfitting.

8 Convolutional Neural Networks: Teaching Computers to See

An ordinary dense layer treats an image as one long list of unrelated numbers, throwing away the fact that nearby pixels belong together. A **Convolutional Neural Network** (CNN) fixes this with a layer built specifically for grids of pixels: the **convolution layer**.

☞ The intuition

A convolution layer slides one small grid of numbers, called a **filter** (or **kernel**), across the whole image. At every position, it does one tiny weighted sum between the filter and the patch of pixels underneath it. The same filter is reused at every position, so it finds the same pattern — an edge, a corner, a texture — no matter where it appears in the image.

★ Everyday picture

Imagine stamping a small rubber stamp, over and over, across a big sheet of paper — left to right, top to bottom — and recording how dark the ink mark is each time. The stamp's pattern never changes, but wherever the paper underneath matches the stamp's shape, you get a strong, dark mark. That is exactly what a convolution filter does to an image, one small patch at a time.

The output-size formula, built up

Suppose the input is $W \times W \times D$ (width, height, depth — e.g. 3 colour channels), the filter size is $F \times F$, the stride is S (how many pixels the filter jumps each step), and P is the padding (extra zero-pixels added around the border). Using D_{out} filters, the output width is

$$W_{out} = \frac{W - F + 2P}{S} + 1,$$

giving an output volume of size $W_{out} \times W_{out} \times D_{out}$.

✎ Worked example: computing convolution output size, then a real convolution

An image is 7×7, the filter is 3×3, and the stride is $S = 1$.

Step 1. No padding ($P = 0$): $W_{out} = (7-3+0)/1+1 = 4+1 = \mathbf{5}$. A 7×7 image shrinks to 5×5.

Step 2. "Same" padding ($P = 1$): $W_{out} = (7-3+2)/1+1 = 6+1 = \mathbf{7}$. With one ring of zero-padding, the output size matches the input — this is why it is called "same" padding.

Step 3. Now the actual convolution. Take one 3×3 image patch and a 3×3 vertical-edge filter, with bias 0.5:

$$\text{patch} = \begin{bmatrix} 1 & 2 & 0 \\ 4 & 1 & 0 \\ 0 & 1 & 3 \end{bmatrix}, \qquad \text{filter} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}.$$

Step 4. Multiply matching cells and add each row. Row 1: $1(1) + 2(0) + 0(-1) = 1$. Row 2: $4(1) + 1(0) + 0(-1) = 4$. Row 3: $0(1) + 1(0) + 3(-1) = -3$.

Step 5. Sum the rows, then add the bias. $1 + 4 - 3 = 2$, so the output pixel is $2 + 0.5 = \mathbf{2.5}$.

Step 6. Sense-check. This single number is one pixel of the output *feature map* — the filter's opinion of "how much does this patch look like a vertical edge?" at this one spot.

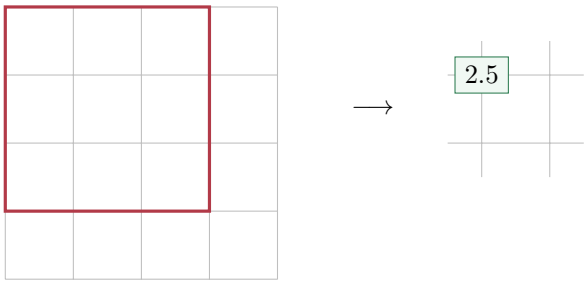


Figure 8. The filter (highlighted patch) slides across the image; each position produces one number in a smaller output grid — here, the 2.5 computed above.

✗ Watch out

Do not confuse the *filter size* F (how big the stamp is) with the *stride* S (how far it jumps between stamps). Also, every real CNN layer uses *many* filters at once, not just one — each learns to detect a different pattern.

Where this shows up in ML. This is precisely `tf.keras.layers.Conv2D(filters=32, kernel_size=3, strides=1, padding='same')`. The learned filters in early layers detect edges; deeper layers combine them into textures, then shapes, then whole objects.

✔ Key takeaway

Convolution slides a small, shared-weight filter across an image, computing one weighted sum per position. The output size follows $W_{out} = (W - F + 2P)/S + 1$.

9 The Pooling Layer: Shrinking Without Losing the Point

☞ The intuition

After a convolution layer produces a large feature map, a **pooling layer** shrinks it by summarising each small neighbourhood down to one number — throwing away the exact position but keeping the strongest signal.

★ Everyday picture

Imagine skimming a book chapter and, for each page, remembering only the single most dramatic sentence — not every word. You end up with a much shorter summary that still captures what mattered most. **Max pooling** does exactly this: for each small patch of the feature map, it keeps only the largest number.

The output-size formula

Given a feature map of size $W \times W \times D$, a pooling window of size $F \times F$, and stride S (usually with no padding), the output size is

$$W_{out} = \frac{W - F}{S} + 1,$$

giving an output of $W_{out} \times W_{out} \times D$ — the depth D never changes; pooling only shrinks width and height.

✎ Worked example: max pooling a 4×4 feature map

A 4×4 feature map is pooled with a 2×2 window and stride $S = 2$.

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 0 & 1 & 8 & 3 \\ 2 & 4 & 1 & 0 \end{bmatrix}$$

Step 1. Output size. $W_{out} = (4-2)/2 + 1 = \mathbf{2}$, so the result is a 2×2 grid.

Step 2. Top-left window $\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}$: the largest value is **6**.

Step 3. Top-right window $\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$: the largest value is **4**.

Step 4. Bottom-left window $\begin{bmatrix} 0 & 1 \\ 2 & 4 \end{bmatrix}$: the largest value is **4**.

Step 5. Bottom-right window $\begin{bmatrix} 8 & 3 \\ 1 & 0 \end{bmatrix}$: the largest value is **8**.

Step 6. Assemble the output. $\begin{bmatrix} 6 & 4 \\ 4 & 8 \end{bmatrix}$ — one quarter the size of the input, keeping each region's strongest signal.

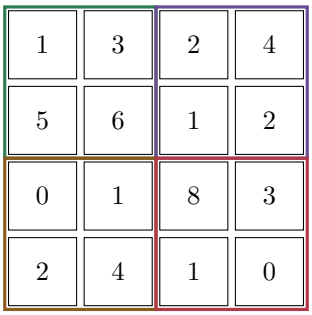


Figure 9. Each coloured 2×2 window keeps only its largest number, shrinking the 4×4 map down to 2×2.

✗ Watch out

Max pooling and *average* pooling are different operations — average pooling keeps the mean of the window instead of the maximum. Also remember pooling shrinks width and height only; the depth (number of channels) stays the same.

Where this shows up in ML. This is `tf.keras.layers.MaxPooling2D(pool_size=2, strides=2)`. Pooling is a big reason CNNs need far fewer parameters than a dense network over raw pixels, and it also makes the model tolerant of the object shifting slightly in the image.

✔ Key takeaway

Pooling shrinks a feature map by keeping only the strongest value in each small window, using $W_{out} = (W - F)/S + 1$, while leaving the depth unchanged.

10 Flattening and the Full Picture: From Pixels to a Prediction

We now have all the pieces: convolution finds patterns, ReLU adds the bend, pooling shrinks the map. The last step turns all of that into a single decision.

☞ The intuition

Flattening takes the final stack of small feature maps — a 3D block of numbers — and unrolls it into one long 1D list. That list is then fed into ordinary dense layers (Section 1) which make the final call, usually finishing with **softmax**, which turns raw scores into probabilities that add up to 1.

★ Everyday picture

Picture unfolding a neatly folded map into one long strip of paper, laid out end to end. No information is lost — it is simply reshaped from a grid into a single line, ready to be read by the dense layers that make the final decision.

Softmax, built step by step

Given k raw output scores $z_1, \ldots, z_k$ (called **logits**), softmax turns them into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}.$$

Every e^{z_j} is positive, so every p_i is positive, and dividing by the total guarantees $\sum_i p_i = 1$ — a genuine probability distribution.

✎ Worked example: from pooled features to a "cat or dog" probability

The last pooling layer outputs a $2 \times 2 \times 3$ block of numbers (2 wide, 2 tall, 3 filters). A dense layer then produces two raw scores (logits): "cat" $= 2.0$, "dog" $= 0.5$. Find the final probabilities.

Step 1. Flatten first. The $2 \times 2 \times 3$ block becomes one list of $2 \times 2 \times 3 = \mathbf{12}$ numbers, fed into the dense layer.

Step 2. Exponentiate each logit. $e^{2.0} = 7.389$, $\quad e^{0.5} = 1.649$.

Step 3. Sum them. $7.389 + 1.649 = 9.038$.

Step 4. Divide each by the total. $p_{\text{cat}} = 7.389/9.038 = \mathbf{0.818}$ (about 81.8%), $\quad p_{\text{dog}} = 1.649/9.038 = \mathbf{0.182}$ (about 18.2%).

Step 5. Sense-check. $0.818 + 0.182 = 1.000$ — the probabilities sum to exactly 1, as softmax always guarantees. The network predicts "cat" with fairly high confidence.

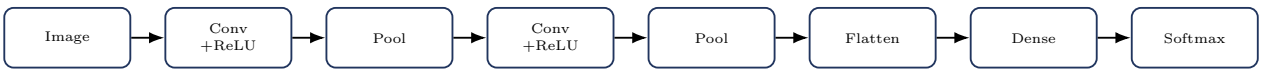


Figure 10. The full CNN pipeline: convolution blocks act as the network's "eyes," while flatten, dense, and softmax act as its final "judgment."

✗ Watch out

Forgetting to flatten before a dense layer is one of the most common shape errors in real code. Always check that softmax's outputs sum to 1 as a sanity check on your own worked examples.

Where this shows up in ML. This exact structure — repeated Conv+ReLU+Pool blocks, then `Flatten()`, then `Dense()`, then softmax — is the architecture behind classic CNNs like LeNet, AlexNet, and VGG, and is the backbone of every image classifier you will build.

✔ **Key takeaway**

Flatten turns the final feature maps into one list of numbers; dense layers and softmax turn that list into a confident, probability-based final answer.

Glossary & symbols

Key terms.

neuron	the basic unit: a weighted sum, plus a bias, pushed through an activation function.
weight	a number saying how much one input matters to a neuron.
bias	a fixed extra number a neuron always adds, shifting its output.
activation function	the rule that bends a neuron's number (e.g. sigmoid, ReLU).
sigmoid	an activation that squashes any number into the range $(0, 1)$.
ReLU	an activation that keeps positive numbers and zeroes out negative ones.
forward propagation	running inputs through a network, layer by layer, to get a prediction.
backpropagation	using the chain rule to find how each weight should change to reduce the error.
gradient descent	repeatedly nudging weights a small step against their gradient to reduce loss.
learning rate	how big a step gradient descent takes each update.
loss function	a number measuring how wrong a prediction was.
overfitting	a model that fits training data very well but fails on new data.
underfitting	a model that fits neither the training data nor new data well.
dropout	randomly switching off neurons during training to avoid over-reliance on any one.
early stopping	halting training once validation performance stops improving.
convolution	sliding a small filter across an image, computing a weighted sum at each position.
filter / kernel	the small grid of learned numbers that a convolution layer slides across the input.
stride	how many pixels a filter moves between each position.
padding	extra zero-pixels added around an image's border before convolving.
feature map	the output grid produced by one convolution filter.
pooling	shrinking a feature map by summarising each small window into one number.
flattening	reshaping a 3D block of feature maps into one 1D list of numbers.
softmax	turning a list of raw scores into probabilities that sum to 1.

Symbols at a glance.

Symbol	Meaning
x_i	one input number to a neuron
w_i	the weight on input x_i
b	the bias, a fixed number added once
z	the weighted sum plus bias, before activation
$f(\cdot),\ \sigma(\cdot)$	the activation function (e.g. sigmoid)
y	a neuron's or network's final output
L	the loss: how wrong a prediction was
η	the learning rate (the Greek letter eta)
W, F, S, P, D	image width, filter size, stride, padding, depth
p	a probability, or dropout's keep probability

Check yourself

✎ Worked example: Q1 — what a neural network is for

Question: What is the primary function of a neural network?

Step 1. Answer: To learn from data and make predictions.

Step 2. Why: A network is not a data store or a calculator — it adjusts its weights from examples (Sections 4–5) so it can predict on new, unseen inputs.

✎ Worked example: Q2 — deep learning vs. traditional machine learning

Question: How is deep learning different from traditional machine learning?

Step 1. Answer: Deep learning models learn useful features straight from raw data; traditional machine learning usually needs a human to hand-craft those features first.

Step 2. Why: A CNN's early convolution layers (Section 8) learn to spot edges on their own — no one tells them to.

✎ Worked example: Q3 — recognising overfitting

Question: What is overfitting?

Step 1. Answer: A model that performs well on training data but poorly on test data.

Step 2. Why: That is exactly Student B's pattern in Section 6: a tiny training error paired with a much larger test error.

✎ Worked example: Q4 — the goal of backpropagation

Question: What is the main goal of backward propagation?

Step 1. Answer: To update the weights based on the error.

Step 2. Why: Section 5 showed this precisely: backprop computes each weight's gradient, and gradient descent then updates it.

✎ Worked example: Q5 — the role of an activation function

Question: What is the role of an activation function in a neural network?

Step 1. Answer: To introduce non-linearity into the model.

Step 2. Why: Section 2 showed that without a bend, stacking layers is still just one straight line — activation functions are what let the network fit curves.